DELIVERYAPPUPDATE·MARCH30

Feature 4: Live Tracking

Bhavya
Feature Owner
30th March 2026

Table of Contents

Quick Recap: What We Had After March 29

After March 29, the entire Feature 4 backend was built and committed. A reminder of what was already in place:

- **TrackingController.java** — handles GET /track/{orderId} (show the page) and POST /track/{orderId}/update (agent clicks a button). Fully built.
- **TrackingService.java** — runs the state machine, saves TrackingEvent records, and tells the page which buttons to show.
- **tracking.html** — the Thymeleaf template: shows order details, the timeline, and agent action buttons. Built, but not yet run live.

Today the goal was to actually run the app and confirm the tracking page works in a browser. That is when we discovered and fixed two real bugs.

What We Did Today: Live Testing & Bug Fixing

To test the tracking page, we needed real data in the database and a way to log in as different users without a login system built yet. We created two temporary helper files:

- **DataLoader.java** — runs once on startup and seeds the database with 3 users (a customer, an agent, and an admin), 1 time slot, and 3 delivery orders at different stages.
- **DevController.java** — adds a shortcut URL: visiting /dev-login?as=agent automatically logs in as the test agent and redirects to /track/1. No real login needed.

With these in place, we ran the app in IntelliJ, confirmed all 5 database tables were auto-created by Hibernate, and opened the tracking page in a browser. The app threw a Thymeleaf error. We traced it, found two bugs in tracking.html, fixed them both, and confirmed the page rendered correctly. Then both temporary files were deleted — they served their purpose and are not part of the final feature.

Bug Fix 1: Status Badge (tracking.html line 155)

What the code does

The status badge at the top of the tracking page is coloured differently depending on the order's current status — green for DELIVERED, red for FAILED, orange for OUT_FOR_DELIVERY, and so on. To pick the right colour, Thymeleaf uses a nested ternary expression: a chain of if/else checks, one for each possible status.

What was wrong

There are 6 possible statuses to check (DELIVERED, FAILED, OUT_FOR_DELIVERY, ASSIGNED, SCHEDULED, RESCHEDULED). Each check opens a bracket. Every opening bracket must be closed. The expression had 6 opening brackets but only 5 closing ones — one was missing at the end. Thymeleaf read the file, ran out of input before finding the last closing bracket, and threw the error: *"Unexpectedly ran out of input"*.

The fix

One closing parenthesis was added at the end of line 155:

Before (5 closing brackets — broken):

```
(s == 'RESCHEDULED' ? 'bg-warning text-dark' : 'bg-secondary')))))}">
```

After (6 closing brackets — correct):

```
(s == 'RESCHEDULED' ? 'bg-warning text-dark' : 'bg-secondary'))))))}">
```

Six opening brackets, six closing brackets. Thymeleaf parsed successfully.

Bug Fix 2: Timeline Dot (tracking.html line 299)

What the code does

Further down the page, the tracking timeline shows a coloured circle (the "timeline dot") next to each status step. This uses the same kind of nested ternary expression to pick the correct Bootstrap colour class.

What was wrong

The same missing-parenthesis bug existed here too — again 6 opens and only 5 closes. In addition, during the first fix attempt, the closing `}` that ends the Thymeleaf `${...}` expression and the `>` that closes the HTML `<div` tag were accidentally removed, requiring two further corrections.

Final correct state

Line 299 — sixth bracket and closing brace restored:

```
(s == 'RESCHEDULED' ? 'bg-warning' : 'bg-secondary'))))))}">
```

Line 300 — the > closes the div tag; the checkmark is the content inside it:

```
>✓
```

After both fixes, Thymeleaf parsed the file without error and the tracking page loaded correctly in the browser.

Three Shared Repositories Added

While setting up the live test, three repository files that had been planned but not yet written were added. These are permanent files needed by Feature 5 (the dashboard) and the login system.

UserRepository.java

The database helper for the users table. Provides two query methods:

- **findByEmail(email)** — used by the login system to look up a user by their email address.
- **findByRole(role)** — used by the admin dashboard to list all agents or all customers.

AgentRepository.java

The database helper for the agents table (agent profile linked to a user account). Provides two query methods:

- **findByUser(user)** — retrieves the agent profile for a specific user. Used when an agent logs in to find their profile.
- **findByAvailableTrue()** — returns all agents currently available to take new orders. Used by Feature 5 (dashboard) and Feature 3 (route optimisation).

TimeSlotRepository.java

The database helper for the time_slots table (scheduled delivery windows). Provides two query methods:

- **findBySlotDateAndAvailableTrue(date)** — returns all open slots on a given date. Used by the booking form (Tanushree's Feature 2) to show customers available windows.
- **findBySlotDate(date)** — returns all slots on a date regardless of availability. Used by admin to see the full schedule.

Cleaning Up: Removing the Test Files

Once the tracking page was confirmed working, DataLoader.java and DevController.java were deleted. These files only existed to support testing — they seeded fake data and bypassed the login system. Leaving them in the codebase would be wrong: they insert test rows every time the app starts and expose a URL that lets anyone log in as any user without a password.

Removing them has no effect on Feature 4. The tracking page, the controller, the service, and the repositories all continue to work exactly as before. The three repositories (UserRepository, AgentRepository, TimeSlotRepository) are permanent and were kept.

Feature 4: Confirmed Complete

All files that make up Feature 4 are correct and working:

- **DeliveryRequestRepository.java** — database queries for orders
- **TrackingEventRepository.java** — fetches the full status-change history for any order
- **TrackingService.java** — state machine, status updates, button logic
- **UserRepository.java** — user lookups (permanent shared file)
- **AgentRepository.java** — agent profile queries (permanent shared file)
- **TimeSlotRepository.java** — time slot queries (permanent shared file)
- **TrackingController.java** — GET and POST endpoints for the tracking page
- **tracking.html** — Thymeleaf template, both ternary bugs now fixed and confirmed working

The app starts cleanly, Hibernate creates all tables, the tracking page renders correctly, and the state machine correctly controls which buttons are shown to the agent.

What's Next

Feature 4 is fully complete and pushed to GitHub. The remaining steps are:

Date	Task
March 30	Push Feature 4 to GitHub (branch: feature-live-tracking) — includes tracking.html fixes, 3 new repositories, and removal of test files
April 1	Begin Feature 5: Admin/Agent Dashboard (branch: feature-dashboard)
April 1+	Dashboard shows agents their assigned orders; admin sees all orders and manages agents
End of project	Bhavya designs and builds the full UI across all pages